

ML-DL-Ops Assignment 3 Report

End-to-End Hugging Face Model Training & Docker Deployment

Dwivedi Jyoti Rajeshbhai
M25CSA010

February 16, 2026

1 Introduction

This assignment implements a complete machine learning workflow using the Hugging Face ecosystem. The pipeline includes:

- Converting given colab notebook into production-ready Python scripts
- Fine-tuning a transformer model
- Evaluating performance
- Containerizing using Docker
- Publishing artifacts to Hugging Face and GitHub
- Fetching model from Hugging Face
- Running downloaded model on local setup and finding evaluation results.

The objective was to build a reproducible and deployment-ready end-to-end ML classification system.

2 Model Selection

The selected base model was:

prajjwal1/bert-tiny

This model is a lightweight version of BERT with:

- 2 transformer layers
- Hidden size: 128
- 2 attention heads
- Max sequence length: 512
- Due to CPU constraint, I had to set `max_length = 128` in `data.py` file for less computation, which led to lower accuracy.

Reason for Selection

- Small model size suitable for Docker deployment
- Fast training time
- Lower computational requirements
- Appropriate for experimentation and pipeline validation

Classification Labels

The model was fine-tuned for 8-class single-label classification:

- romance
- history_biography
- comics_graphic
- poetry
- fantasy_paranormal
- mystery_thriller_crime
- children
- young_adult

3 Environment Setup

Docker was used to ensure reproducibility.

3.1 Dependencies

The following major dependencies were used:

- torch==2.1.2
- transformers==4.36.2
- datasets==2.16.1
- scikit-learn==1.3.2
- accelerate==0.25.0

3.2 Docker Strategy

Two Docker images were created:

- **Training Image** – used for model fine-tuning
- **Evaluation Image** – pulls model from Hugging Face and runs evaluation

This separation ensures clean production deployment.

3.3 Commands:

```
docker build -t assignment_3_jyoti -f Dockerfile.train .
```

```
docker run assignment_3_eval python /src/train.py
```

```
docker build -t assignment_3_eval -f Dockerfile.eval .
```

```
docker run HF_TOKEN="*****" assignment_3_eval python /src/evaluate.py
```

4 Colab Notebook to Scripts conversion

- Project/
 - data.py
 - train.py
 - evaluate.py
 - requirements.txt
 - Dockerfile
 - Dockerfile.train
 - Dockerfile.eval

5 Training Procedure

Training was performed using the Hugging Face `Trainer` API.

5.1 Training Configuration

- Problem Type: Single-label classification
- Optimizer: AdamW (default in Trainer)
- Loss Function: CrossEntropyLoss
- Evaluation Strategy: Per epoch
- Metrics: Accuracy and Weighted F1-score

The Trainer API simplified:

- Automatic logging
- Checkpoint saving
- Metric tracking

6 Evaluation Results

6.1 Initial Evaluation (Before Proper Fine-Tuning)

Metric	Value
Evaluation Loss	3.0326
Accuracy	0.0744
Weighted F1	0.0695

Table 1: Initial Evaluation Results

The model initially performed poorly, close to random guessing across 8 classes.

6.2 Final Evaluation (After Training)

Metric	Value
Evaluation Loss	2.0885
Accuracy	0.1025
Weighted F1	0.1276

Table 2: Final Evaluation Results

6.3 Performance Improvement

- Accuracy improved from 7.4% to 10.25%
- Weighted F1 improved from 0.0695 to 0.1276
- Significant reduction in evaluation loss

This demonstrates successful fine-tuning.

6.4 Why Accuracy Is Low

The relatively low accuracy (10.25%) can be attributed to:

- Reduced max sequence length (128 instead of 512)
- Limited model capacity (2-layer BERT-Tiny)
- Potential class imbalance
- Insufficient training epochs

7 Class-wise Observations

- Strong performance for: `comics_graphic`, `poetry`
- Moderate performance for: `romance`, `mystery_thriller_crime`
- Weak performance for: `history_biography`, `children`, `young_adult`

Possible reasons:

- Class imbalance
- Semantic overlap between categories
- Limited capacity of BERT-Tiny

8 Model Deployment

The fine-tuned model was:

- Saved locally
- Uploaded to Hugging Face Hub
 - `model.push_to_hub("JD16112001/model_assignment3")`
 - `tokenizer.push_to_hub("JD16112001/model_assignment3")`
- The hugging face repository has:
 - `.gitattributes`
 - `config.json`
 - `model.safetensors`
 - `training_args.bin`
- Reloaded from Hugging Face repository
- Evaluated again inside Docker container

Metrics matched local evaluation, confirming deployment consistency.

9 Production Docker Image

The final Docker image:

- Pulls model from Hugging Face
- Automatically runs evaluation on startup
- Ensures reproducibility
- Contains minimal dependencies

This simulates a production inference environment.

10 Important Links

- GitHub: <https://github.com/Jyoti-Dwivedi-010/MLOps-Jyoti-Dwivedi-M25CSA010/tree/Assignment-3>
- HuggingFace: https://huggingface.co/JD16112001/model_assignment3

11 Challenges Faced

- Converting notebook to modular scripts
- Managing dependencies inside Docker
- Ensuring GPU compatibility
- Debugging evaluation mismatches
- Hugging Face authentication inside container

12 Conclusion

This assignment successfully demonstrated:

- End-to-end ML workflow development
- Transformer fine-tuning
- Model evaluation and analysis
- Docker-based reproducibility
- Cloud model hosting using Hugging Face

Although BERT-Tiny achieved low performance (10.25% accuracy), the primary objective of building a production-ready ML pipeline was successfully accomplished.

The improvements that I can try, includes:

- Using larger models (e.g., BERT-base)
- Hyperparameter tuning
- Early stopping
- Increase training epochs
- Data augmentation
- Class rebalancing techniques